

Exercise 2 : Dismounting sequence

Task 1: Algorithm

```
List<E> dismounting(Graph<E> graph):  
    visited ← set  
    dismountOrder ← list  
  
    for node in graph:  
        if node not in visited:  
            DFS_recur(node, graph, visited, dismountOrder)  
  
    return removeOrder
```

In the algorithm we design for `dismounting`, we:

1. Create an empty set `visited` to store the nodes already been visited.

Create an empty list `dismountOrder` to store the result list.

2. Use **DFS** to traverse the graph

```
void DFS_recur(node, graph, visited, order):  
    add node to visited  
    for nextNode in neighbors(node):  
        if nextNode not in visited:  
            DFS_recur(nextNode, graph, visited, order)  
    add node to order
```

- Mark current node as visited if it is not.
- For each node depends on this, if it is not visited, traverse by DFS
- After all children are explored, add node to the list `dismountOrder`

3. Return the list `dismountOrder`. Now the length of the list should equal to the size of the set `visited`.

Task 2: Complexity Analysis

The dismounting algorithm visits each vertex and each edge exactly once using DFS. And the other operations, like conditional check, all run in $O(1)$. Therefore, the worst-case time complexity is the same as DFS:

$$O(V + E)$$

, where V is the number of vertices (components),

E is the number of edges (dependency relations).

Task 3: Implementation

[code in the wet part : `DismountOrder`]